Klasifikasi Citra MNIST

Langkah 1: Persiapan Dataset MNIST dan DataLoader

Pada tahapan ini, kita melakukan:

1. Pengecekan akselerator GPU (CUDA) yang aktif.
2. Pembagian ulang dataset MNIST mentah secara deterministik dengan rasio **80% Data Latih** dan **20% Data Uji**.
3. Normalisasi skala piksel dan transformasi dimensi ke standar format tensor PyTorch `[N, C, H, W]`.
4. Konversi data ke dalam objek `TensorDataset` dan pembentukan `DataLoader` untuk mengoptimalkan transfer batch memori ke GPU.

In [1]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import tensorflow as tf
import itertools
import random

# Atur seed acak agar hasil eksperimen konsisten
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)

# Tentukan device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Perangkat aktif untuk PyTorch: {device}")

# Load dataset menggunakan helper TensorFlow untuk kesederhanaan
(train_images_raw, train_labels_raw), (test_images_raw, test_labels_raw) = tf.ke

# Gabungkan dan bagi ulang dengan rasio 80:20
all_images = np.concatenate((train_images_raw, test_images_raw), axis=0)
all_labels = np.concatenate((train_labels_raw, test_labels_raw), axis=0)

# Normalisasi dan transpose agar dimensi sesuai dengan PyTorch [N, C, H, W]
all_images = all_images.reshape((-1, 1, 28, 28)).astype('float32') / 255.0

X_train_np, X_test_np, y_train_np, y_test_np = train_test_split(
    all_images, all_labels, test_size=0.20, random_state=42
)

# Konversi ke Tensor PyTorch
X_train_pt = torch.tensor(X_train_np, dtype=torch.float32)
```

```
y_train_pt = torch.tensor(y_train_np, dtype=torch.long)
X_test_pt = torch.tensor(X_test_np, dtype=torch.float32)
y_test_pt = torch.tensor(y_test_np, dtype=torch.long)

# Loader Data
train_dataset = TensorDataset(X_train_pt, y_train_pt)
test_dataset = TensorDataset(X_test_pt, y_test_pt)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

print(f"Tensor Data Latih: {X_train_pt.shape}")
print(f"Tensor Data Uji: {X_test_pt.shape}")
```

```
Perangkat aktif untuk PyTorch: cuda
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-dataset
s/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
Tensor Data Latih: torch.Size([56000, 1, 28, 28])
Tensor Data Uji: torch.Size([14000, 1, 28, 28])
```

Langkah 2: Definisi Arsitektur Model CNN (Convolutional Neural Network)

Kita merancang kelas arsitektur CNN kustom menggunakan `nn.Module` dari PyTorch dengan spesifikasi sebagai berikut:

- **Blok Konvolusi**: Dua layer konvolusi `nn.Conv2d` diikuti aktivasi ReLU dan Pooling `nn.MaxPool2d` untuk ekstraksi fitur spasial.
- **Blok Klasifikasi (Dense)**: Pendataran dimensi fitur (`nn.Flatten`), layer fully-connected (`nn.Linear`) dengan parameter jumlah unit dinamis, regularisasi `Dropout` untuk mencegah *overfitting*, dan output layer proyeksi ke-10 kelas digit.

In [2]:
```
class MNISTPyTorchCNN(nn.Module):
    def __init__(self, dense_units=64, activation_type='relu', dropout_rate=0.25
        super(MNISTPyTorchCNN, self).__init__()

        # Tentukan fungsi aktivasi sesuai parameter
        if activation_type == 'relu':
            self.activation = nn.ReLU()
        elif activation_type == 'leaky_relu':
            self.activation = nn.LeakyReLU(0.1)
        elif activation_type == 'tanh':
            self.activation = nn.Tanh()
        elif activation_type == 'softmax':
            self.activation = nn.Softmax(dim=1)
        else:
            self.activation = nn.ReLU()

        self.conv_layers = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=0),
            self.activation,
            nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, kernel_size=3, padding=0),
            self.activation,
            nn.MaxPool2d(2, 2)
```

```
        )
        self.flatten = nn.Flatten()
        self.fc_layers = nn.Sequential(
            nn.Linear(5 * 5 * 64, dense_units),
            self.activation,
            nn.Dropout(dropout_rate),
            nn.Linear(dense_units, 10)
        )

    def forward(self, x):
        x = self.conv_layers(x)
        x = self.flatten(x)
        x = self.fc_layers(x)
        return x
```

Langkah 3: Eksekusi Pelatihan Multi-Konfigurasi Hyperparameter pada CUDA GPU

Di bawah ini, kita melakukan pengujian performa komparatif terhadap 4 kombinasi konfigurasi hyperparameter berbeda:

1. Optimizer **Adam** vs **SGD (with Momentum)**.
2. Jumlah kapasitas unit pada Dense Layer (**64** unit vs **128** unit).

Semua kalkulasi beban kerja pelatihan, pembaruan gradien, dan evaluasi pengujian dijalankan langsung di dalam memori CUDA GPU.

In [3]:
```
# Definisikan semua grid search space dari kombinasi parameter
activations = ['relu', 'leaky_relu', 'tanh', 'softmax']
optimizers = ['adam', 'sgd']
dense_units_list = [64, 128]

criterion = nn.CrossEntropyLoss()
pt_models = {}
pt_results = []
pt_histories = {}  # Kamus untuk menyimpan riwayat training dan validasi

# Lakukan kombinasi produk kartesian dari seluruh parameter
combinations = list(itertools.product(activations, optimizers, dense_units_list))

print(f"Memulai pelatihan untuk {len(combinations)} kombinasi model kustom...")

for act, opt, units in combinations:
    config_name = f"{act.upper()} - {opt.upper()} - {units} Units"
    print(f"\n--- Melatih: {config_name} ---")

    # Inisialisasi model dan kirim ke device GPU
    model = MNISTPyTorchCNN(dense_units=units, activation_type=act).to(device)

    if opt == 'adam':
        optimizer = optim.Adam(model.parameters(), lr=0.001)
    else:
        optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

    history = {'train_loss': [], 'val_loss': [], 'val_acc': []}
```

```
    # Melatih selama 3 epoch agar grafik perkembangan performa terlihat lebih in
    epochs = 3
    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for inputs, targets in train_loader:
            inputs, targets = inputs.to(device), targets.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, targets)
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * inputs.size(0)

        epoch_train_loss = running_loss / len(train_loader.dataset)

        # Evaluasi pada akhir epoch
        model.eval()
        correct = 0
        total = 0
        epoch_val_loss = 0.0
        with torch.no_grad():
            for inputs, targets in test_loader:
                inputs, targets = inputs.to(device), targets.to(device)
                outputs = model(inputs)
                loss = criterion(outputs, targets)
                epoch_val_loss += loss.item() * inputs.size(0)
                _, predicted = torch.max(outputs, 1)
                total += targets.size(0)
                correct += (predicted == targets).sum().item()

        epoch_val_acc = (correct / total) * 100
        epoch_val_loss = epoch_val_loss / total

        history['train_loss'].append(epoch_train_loss)
        history['val_loss'].append(epoch_val_loss)
        history['val_acc'].append(epoch_val_acc)

        print(f"Epoch {epoch+1}/{epochs} | Train Loss: {epoch_train_loss:.4f} |

    pt_models[config_name] = model
    pt_histories[config_name] = history

    pt_results.append({
        "Configuration": config_name,
        "Activation": act.upper(),
        "Optimizer": opt.upper(),
        "Dense Units": units,
        "Test Accuracy": f"{history['val_acc'][-1]:.2f}%",
        "Test Loss": f"{history['val_loss'][-1]:.4f}"
    })

df_pt_results = pd.DataFrame(pt_results)
display(df_pt_results)
```

```
Memulai pelatihan untuk 16 kombinasi model kustom...

--- Melatih: RELU - ADAM - 64 Units ---
Epoch 1/3 | Train Loss: 0.2791 | Val Loss: 0.0763 | Val Acc: 97.71%
Epoch 2/3 | Train Loss: 0.0912 | Val Loss: 0.0584 | Val Acc: 98.26%
Epoch 3/3 | Train Loss: 0.0682 | Val Loss: 0.0540 | Val Acc: 98.27%

--- Melatih: RELU - ADAM - 128 Units ---
Epoch 1/3 | Train Loss: 0.2176 | Val Loss: 0.0667 | Val Acc: 98.01%
Epoch 2/3 | Train Loss: 0.0669 | Val Loss: 0.0563 | Val Acc: 98.09%
Epoch 3/3 | Train Loss: 0.0515 | Val Loss: 0.0413 | Val Acc: 98.61%

--- Melatih: RELU - SGD - 64 Units ---
Epoch 1/3 | Train Loss: 0.4515 | Val Loss: 0.1064 | Val Acc: 96.61%
Epoch 2/3 | Train Loss: 0.1134 | Val Loss: 0.0715 | Val Acc: 97.79%
Epoch 3/3 | Train Loss: 0.0823 | Val Loss: 0.0528 | Val Acc: 98.23%

--- Melatih: RELU - SGD - 128 Units ---
Epoch 1/3 | Train Loss: 0.3703 | Val Loss: 0.0957 | Val Acc: 97.14%
Epoch 2/3 | Train Loss: 0.0933 | Val Loss: 0.0673 | Val Acc: 97.79%
Epoch 3/3 | Train Loss: 0.0690 | Val Loss: 0.0587 | Val Acc: 98.19%

--- Melatih: LEAKY_RELU - ADAM - 64 Units ---
Epoch 1/3 | Train Loss: 0.2088 | Val Loss: 0.0684 | Val Acc: 97.82%
Epoch 2/3 | Train Loss: 0.0652 | Val Loss: 0.0475 | Val Acc: 98.51%
Epoch 3/3 | Train Loss: 0.0496 | Val Loss: 0.0441 | Val Acc: 98.52%

--- Melatih: LEAKY_RELU - ADAM - 128 Units ---
Epoch 1/3 | Train Loss: 0.1804 | Val Loss: 0.0616 | Val Acc: 98.10%
Epoch 2/3 | Train Loss: 0.0548 | Val Loss: 0.0488 | Val Acc: 98.42%
Epoch 3/3 | Train Loss: 0.0399 | Val Loss: 0.0421 | Val Acc: 98.74%

--- Melatih: LEAKY_RELU - SGD - 64 Units ---
Epoch 1/3 | Train Loss: 0.4034 | Val Loss: 0.0953 | Val Acc: 97.15%
Epoch 2/3 | Train Loss: 0.0978 | Val Loss: 0.0760 | Val Acc: 97.69%
Epoch 3/3 | Train Loss: 0.0722 | Val Loss: 0.0650 | Val Acc: 98.00%

--- Melatih: LEAKY_RELU - SGD - 128 Units ---
Epoch 1/3 | Train Loss: 0.3662 | Val Loss: 0.0914 | Val Acc: 97.09%
Epoch 2/3 | Train Loss: 0.0873 | Val Loss: 0.0676 | Val Acc: 98.01%
Epoch 3/3 | Train Loss: 0.0606 | Val Loss: 0.0502 | Val Acc: 98.40%

--- Melatih: TANH - ADAM - 64 Units ---
Epoch 1/3 | Train Loss: 0.2130 | Val Loss: 0.0694 | Val Acc: 97.84%
Epoch 2/3 | Train Loss: 0.0648 | Val Loss: 0.0520 | Val Acc: 98.56%
Epoch 3/3 | Train Loss: 0.0519 | Val Loss: 0.0469 | Val Acc: 98.56%

--- Melatih: TANH - ADAM - 128 Units ---
Epoch 1/3 | Train Loss: 0.1779 | Val Loss: 0.0638 | Val Acc: 97.98%
Epoch 2/3 | Train Loss: 0.0572 | Val Loss: 0.0458 | Val Acc: 98.54%
Epoch 3/3 | Train Loss: 0.0435 | Val Loss: 0.0452 | Val Acc: 98.66%

--- Melatih: TANH - SGD - 64 Units ---
Epoch 1/3 | Train Loss: 0.3607 | Val Loss: 0.1086 | Val Acc: 96.74%
Epoch 2/3 | Train Loss: 0.0933 | Val Loss: 0.0720 | Val Acc: 97.76%
Epoch 3/3 | Train Loss: 0.0649 | Val Loss: 0.0714 | Val Acc: 97.91%

--- Melatih: TANH - SGD - 128 Units ---
Epoch 1/3 | Train Loss: 0.3395 | Val Loss: 0.1030 | Val Acc: 96.97%
Epoch 2/3 | Train Loss: 0.0812 | Val Loss: 0.0690 | Val Acc: 97.86%
```

```
Epoch 3/3 | Train Loss: 0.0593 | Val Loss: 0.0563 | Val Acc: 98.12%

--- Melatih: SOFTMAX - ADAM - 64 Units ---
Epoch 1/3 | Train Loss: 1.9219 | Val Loss: 1.3598 | Val Acc: 64.51%
Epoch 2/3 | Train Loss: 1.2931 | Val Loss: 0.9967 | Val Acc: 70.04%
Epoch 3/3 | Train Loss: 1.1479 | Val Loss: 0.8490 | Val Acc: 70.36%

--- Melatih: SOFTMAX - ADAM - 128 Units ---
Epoch 1/3 | Train Loss: 1.9794 | Val Loss: 1.4158 | Val Acc: 59.39%
Epoch 2/3 | Train Loss: 1.3214 | Val Loss: 1.0164 | Val Acc: 61.71%
Epoch 3/3 | Train Loss: 1.1859 | Val Loss: 0.8833 | Val Acc: 62.11%

--- Melatih: SOFTMAX - SGD - 64 Units ---
Epoch 1/3 | Train Loss: 2.3018 | Val Loss: 2.3014 | Val Acc: 11.43%
Epoch 2/3 | Train Loss: 2.3016 | Val Loss: 2.3011 | Val Acc: 11.43%
Epoch 3/3 | Train Loss: 2.3017 | Val Loss: 2.3008 | Val Acc: 11.43%

--- Melatih: SOFTMAX - SGD - 128 Units ---
Epoch 1/3 | Train Loss: 2.3017 | Val Loss: 2.3007 | Val Acc: 11.43%
Epoch 2/3 | Train Loss: 2.3016 | Val Loss: 2.3011 | Val Acc: 11.43%
Epoch 3/3 | Train Loss: 2.3016 | Val Loss: 2.3015 | Val Acc: 11.43%
```

	Configuration	Activation	Optimizer	Dense Units	Test Accuracy	Test Loss
0	RELU - ADAM - 64 Units	RELU	ADAM	64	98.27%	0.0540
1	RELU - ADAM - 128 Units	RELU	ADAM	128	98.61%	0.0413
2	RELU - SGD - 64 Units	RELU	SGD	64	98.23%	0.0528
3	RELU - SGD - 128 Units	RELU	SGD	128	98.19%	0.0587
4	LEAKY_RELU - ADAM - 64 Units	LEAKY_RELU	ADAM	64	98.52%	0.0441
5	LEAKY_RELU - ADAM - 128 Units	LEAKY_RELU	ADAM	128	98.74%	0.0421
6	LEAKY_RELU - SGD - 64 Units	LEAKY_RELU	SGD	64	98.00%	0.0650
7	LEAKY_RELU - SGD - 128 Units	LEAKY_RELU	SGD	128	98.40%	0.0502
8	TANH - ADAM - 64 Units	TANH	ADAM	64	98.56%	0.0469
9	TANH - ADAM - 128 Units	TANH	ADAM	128	98.66%	0.0452
10	TANH - SGD - 64 Units	TANH	SGD	64	97.91%	0.0714
11	TANH - SGD - 128 Units	TANH	SGD	128	98.12%	0.0563
12	SOFTMAX - ADAM - 64 Units	SOFTMAX	ADAM	64	70.36%	0.8490
13	SOFTMAX - ADAM - 128 Units	SOFTMAX	ADAM	128	62.11%	0.8833
14	SOFTMAX - SGD - 64 Units	SOFTMAX	SGD	64	11.43%	2.3008
15	SOFTMAX - SGD - 128 Units	SOFTMAX	SGD	128	11.43%	2.3015

In [4]:
```
# Memastikan variabel pt_histories tersedia sebelum plotting untuk menghindari N
if 'pt_histories' in globals() and pt_histories:
    fig, axes = plt.subplots(2, 1, figsize=(14, 12))

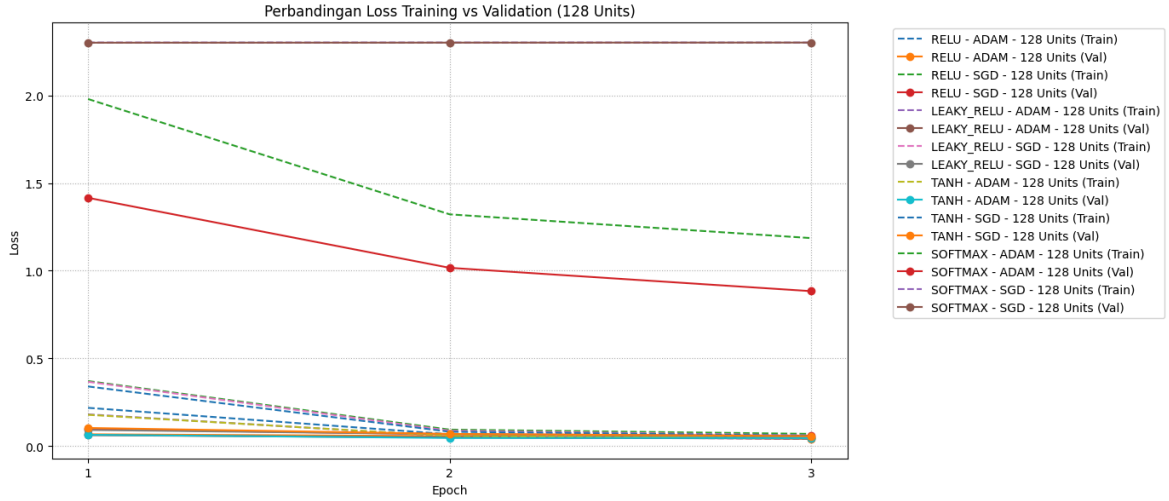
    # Gambar kurva Loss
    for config_name, history in pt_histories.items():
        if '128' in config_name:
            axes[0].plot(range(1, 4), history['train_loss'], label=f"{config_nam
            axes[0].plot(range(1, 4), history['val_loss'], label=f"{config_name}

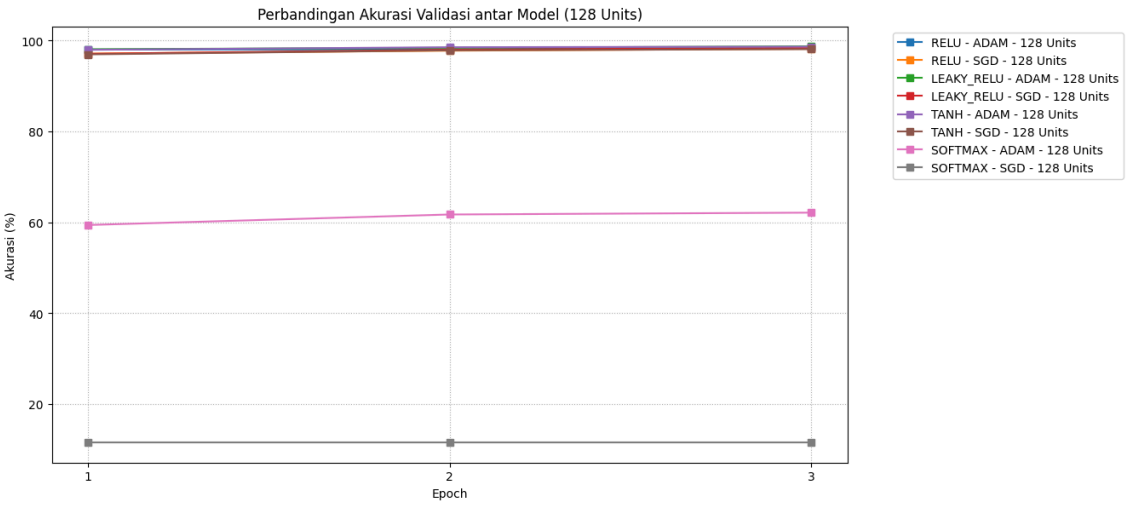
    axes[0].set_title('Perbandingan Loss Training vs Validation (128 Units)')
    axes[0].set_xlabel('Epoch')
    axes[0].set_ylabel('Loss')
    axes[0].set_xticks([1, 2, 3])
    axes[0].legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    axes[0].grid(True, linestyle=':')
```

```
    # Gambar kurva Akurasi Validasi
    for config_name, history in pt_histories.items():
        if '128' in config_name:
            axes[1].plot(range(1, 4), history['val_acc'], label=f"{config_name}"

    axes[1].set_title('Perbandingan Akurasi Validasi antar Model (128 Units)')
    axes[1].set_xlabel('Epoch')
    axes[1].set_ylabel('Akurasi (%)')
    axes[1].set_xticks([1, 2, 3])
    axes[1].legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    axes[1].grid(True, linestyle=':')

    plt.tight_layout()
    plt.show()
else:
    print("Silakan jalankan cell Langkah 3 terlebih dahulu untuk melatih model d
```



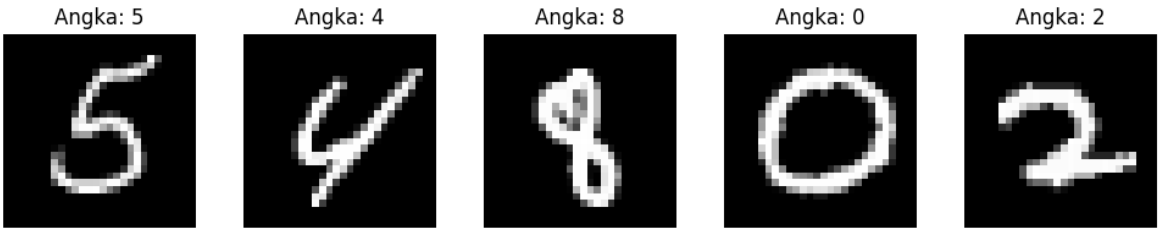


Langkah 5: Visualisasi Grafik Perbandingan Metrik Training vs Validasi

Di bawah ini kita memvisualisasikan grafik kurva loss latihan (*training*) dan loss pengujian (*validation*) untuk melihat karakteristik konvergensi dan pola overfitting dari masing-masing kombinasi arsitektur.

In [5]:
```
# Visualisasi data dari PyTorch Tensor
plt.figure(figsize=(10, 2))
for i in range(5):
```

```
    plt.subplot(1, 5, i+1)
    # Mengubah format tensor [C, H, W] kembali ke [H, W] untuk visualisasi matpl
    plt.imshow(X_train_pt[i].squeeze().numpy(), cmap='gray')
    plt.title(f"Angka: {y_train_pt[i].item()}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```



Langkah 6: Inferensi dan Klasifikasi Langsung Menggunakan Sampel Baru

Sel di bawah ini mengambil beberapa contoh acak dari dataset uji (`X_test_pt`), mengirimkannya ke GPU CUDA (jika tersedia), lalu menampilkan prediksi dari semua model PyTorch yang sudah kita latih.

In [7]:
```
# Pilih 5 indeks acak dari dataset uji secara dinamis
random.seed(42)
sample_indices = random.sample(range(len(X_test_pt)), 5)

# Ambil gambar dan label, kirimkan ke device aktif (CUDA/CPU)
imgs_to_test = X_test_pt[sample_indices].to(device)
true_labels = y_test_pt[sample_indices]

# Evaluasi menggunakan model-model PyTorch terlatih
for name, model in pt_models.items():
    model.eval()
    with torch.no_grad():
        logits = model(imgs_to_test)
        _, preds = torch.max(logits, 1)
        preds = preds.cpu().numpy()

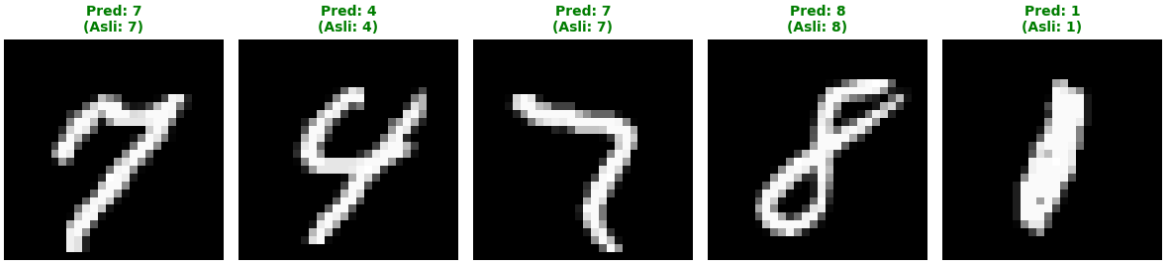
    fig, axes = plt.subplots(1, 5, figsize=(12, 3))
    fig.suptitle(f"Hasil Prediksi Model: {name}", fontsize=14, fontweight='bold'

    for i, idx in enumerate(sample_indices):
        # Render gambar menggunakan matplotlib
        axes[i].imshow(X_test_pt[idx].squeeze().numpy(), cmap='gray')
        actual = true_labels[i].item()
        predicted = preds[i]

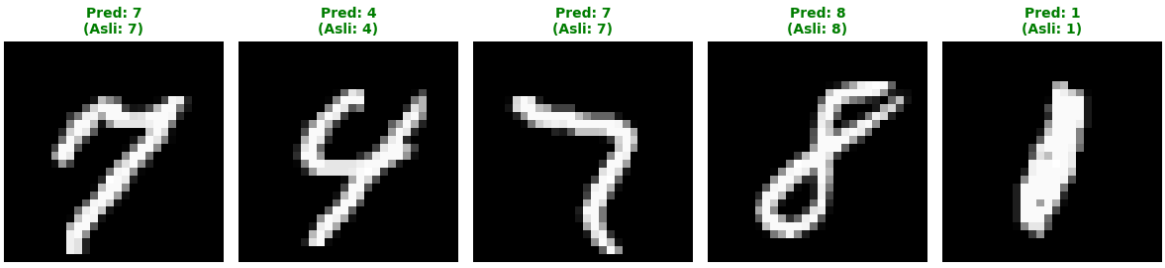
        # Beri warna hijau jika prediksi benar, merah jika salah
        color = 'green' if actual == predicted else 'red'
        axes[i].set_title(f"Pred: {predicted}\n(Asli: {actual})", color=color, f
        axes[i].axis('off')

    plt.tight_layout()
    plt.show()
```

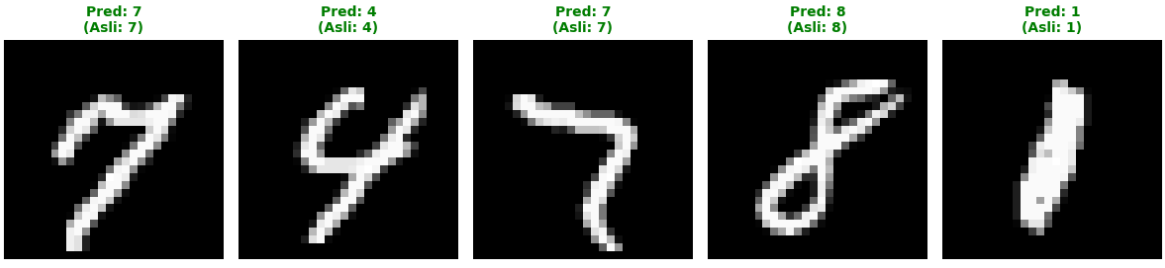
Hasil Prediksi Model: RELU - ADAM - 64 Units



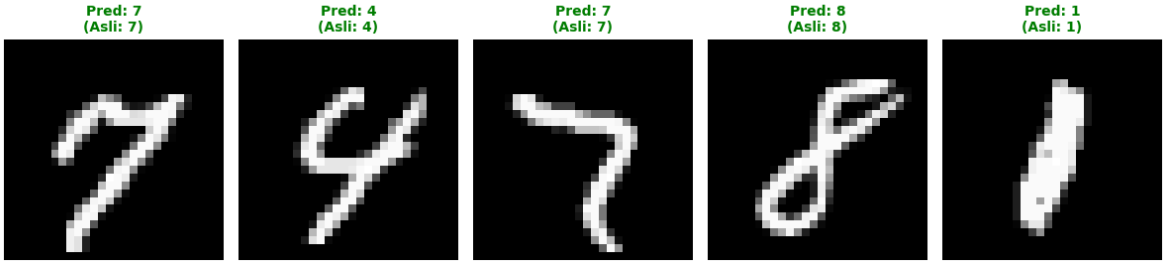
Hasil Prediksi Model: RELU - ADAM - 128 Units



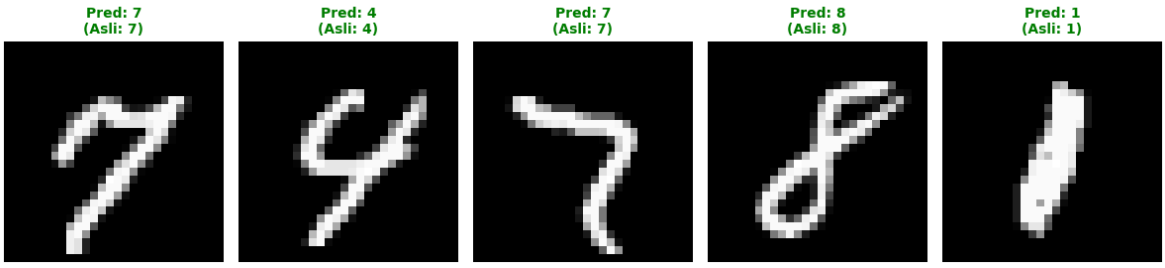
Hasil Prediksi Model: RELU - SGD - 64 Units



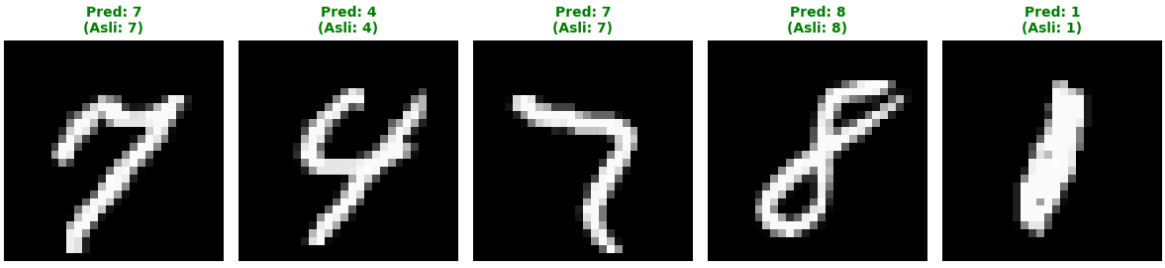
Hasil Prediksi Model: RELU - SGD - 128 Units



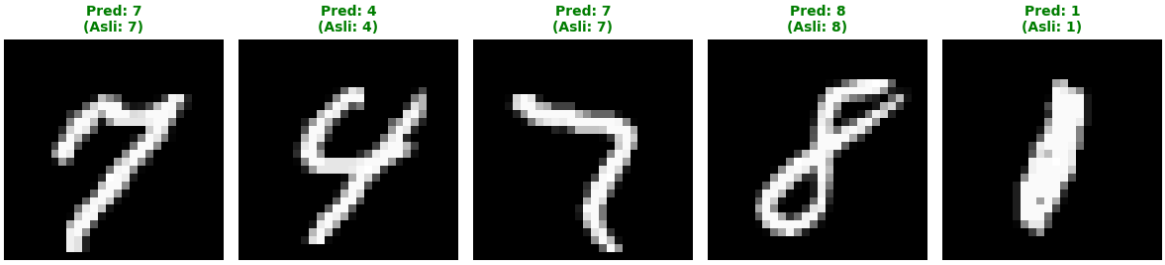
Hasil Prediksi Model: LEAKY_RELU - ADAM - 64 Units



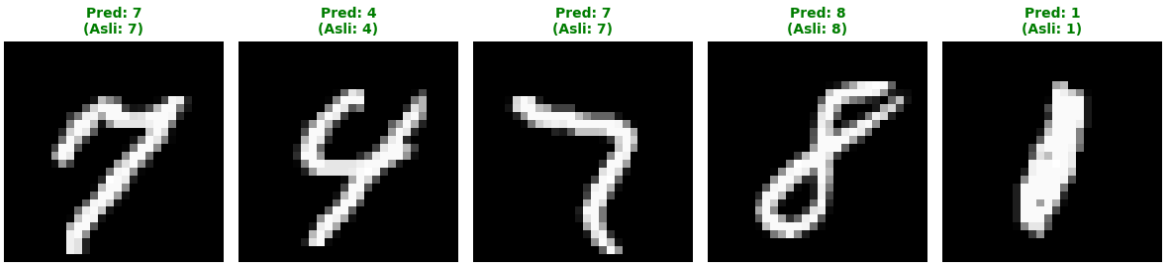
Hasil Prediksi Model: LEAKY_RELU - ADAM - 128 Units



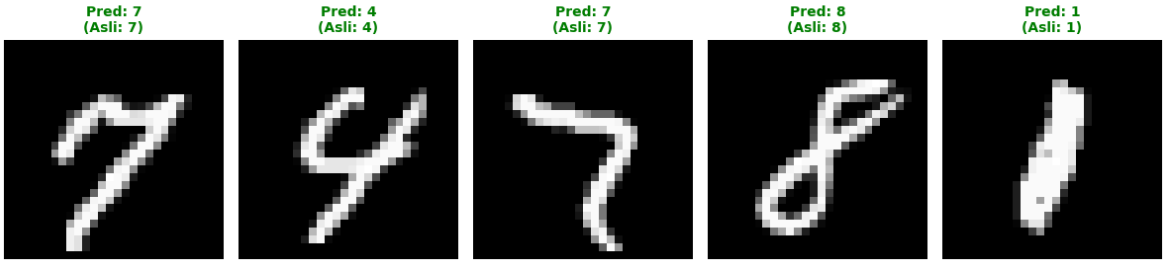
Hasil Prediksi Model: LEAKY_RELU - SGD - 64 Units



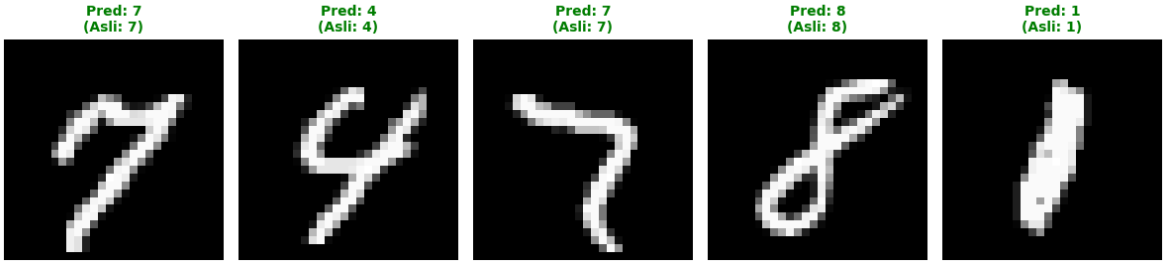
Hasil Prediksi Model: LEAKY_RELU - SGD - 128 Units



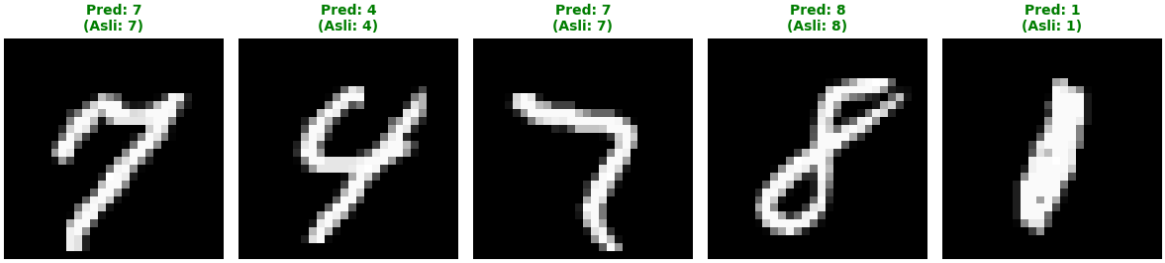
Hasil Prediksi Model: TANH - ADAM - 64 Units



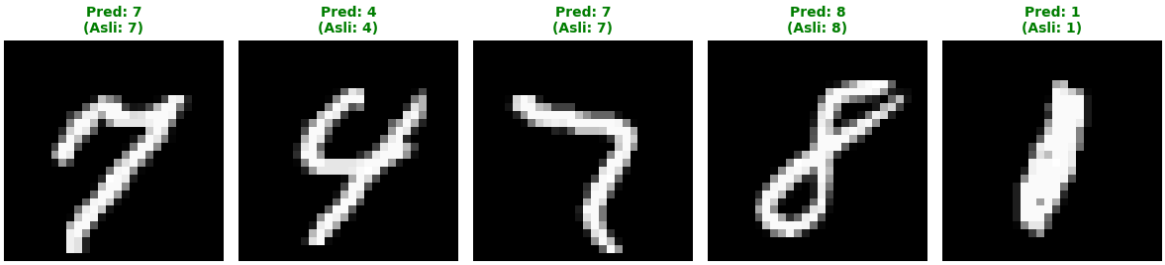
Hasil Prediksi Model: TANH - ADAM - 128 Units

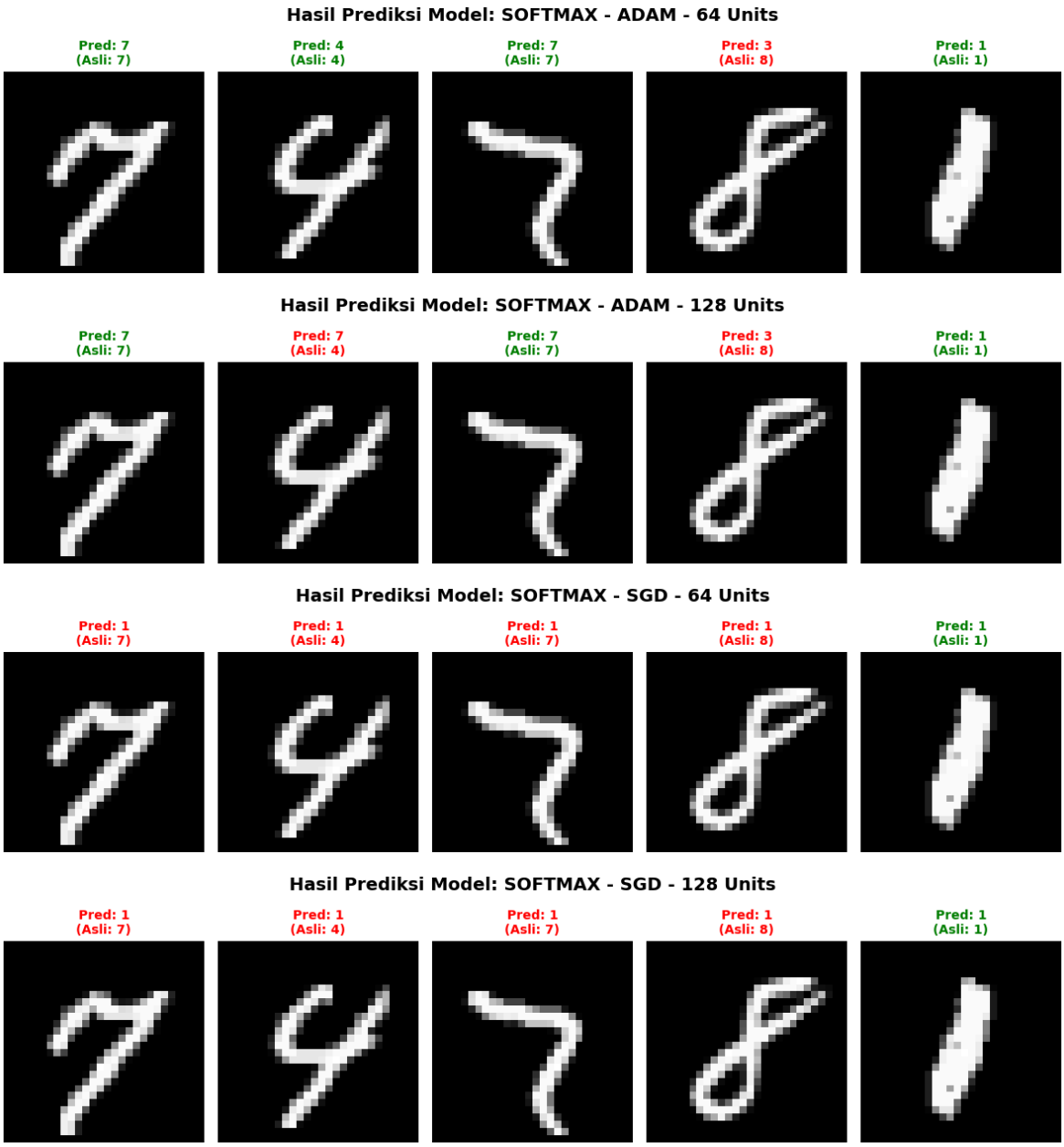


Hasil Prediksi Model: TANH - SGD - 64 Units



Hasil Prediksi Model: TANH - SGD - 128 Units




Hasil Prediksi Model: SOFTMAX - ADAM - 64 Units
Pred: 7 (Asli: 7)
Pred: 4 (Asli: 4)
Pred: 7 (Asli: 7)
Pred: 3 (Asli: 8)
Pred: 1 (Asli: 1)
Hasil Prediksi Model: SOFTMAX - ADAM - 128 Units
Pred: 7 (Asli: 7)
Pred: 7 (Asli: 4)
Pred: 7 (Asli: 7)
Pred: 3 (Asli: 8)
Pred: 1 (Asli: 1)
Hasil Prediksi Model: SOFTMAX - SGD - 64 Units
Pred: 1 (Asli: 7)
Pred: 1 (Asli: 4)
Pred: 1 (Asli: 7)
Pred: 1 (Asli: 8)
Pred: 1 (Asli: 1)
Hasil Prediksi Model: SOFTMAX - SGD - 128 Units
Pred: 1 (Asli: 7)
Pred: 1 (Asli: 4)
Pred: 1 (Asli: 7)
Pred: 1 (Asli: 8)
Pred: 1 (Asli: 1)

In [ ]: